PROJECT 2 OBJECTIVE

In this project we wanted to design a simple calculator that can do addition subtraction multiplication and division for 4 bit numbers another goal was to simulate our design and see if it works correctly in ModelSim and at the end we wanted to understand better how real digital systems work inside by building a small one ourselves

INTRODUCTION to PROJECT 2

In this project, we used Structural Verilog, ModelSim

Structural Verilog:

1. Used to describe the circuit by connecting different components in a modular way.
2 Helped in creating a representation of the design.

ModelSim:

1. Used for simulating the Verilog code to verify the circuit's functionality.
2. Allowed us to analyze waveform outputs and debug any issues before implementation.

Our Verilog CODES

Inside of **project2** file:

1) decoder_1to2.v :

```
module decoder_1to2 (
    input wire write_addr,
    output reg [1:0] sel
);
    always @(*) begin
        sel = (write_addr == 1'b0) ? 2'b01 : 2'b10;
    end
endmodule
```

---

2) tb_decoder_1to2.v :

```
module tb_decoder_1to2;
    reg write_addr;
    wire [1:0] sel;

    decoder_1to2 uut (
        .write_addr(write_addr),
        .sel(sel)
    );

    initial begin
        $monitor("Time=%0t | write_addr = %b => sel = %b", $time, write_addr, sel);

        write_addr = 0; #10;
        write_addr = 1; #10;

        $finish;
    end
endmodule
```

3) mux2to1.v :

```
module mux2to1 (
    input wire [3:0] in0, in1,
    input wire sel,
    output wire [3:0] out
);
    assign out = (sel == 0) ? in0 : in1;
endmodule
```

---

4) tb_mux2to1.v :

```
module tb_mux2to1;
    reg [3:0] in0, in1;
    reg sel;
    wire [3:0] out;

    mux2to1 uut (
        .in0(in0), .in1(in1), .sel(sel), .out(out)
    );

    initial begin
        $monitor("Time=%0t | sel = %b | in0 = %b | in1 = %b => out = %b", $time, sel, in0, in1, out);

        in0 = 4'b1100; in1 = 4'b0011;
        sel = 0; #10;
        sel = 1; #10;

        $finish;
    end
endmodule
```

---

5) mux2to1_1bit.v:

```
module mux2to1_1bit (
    input wire in0, in1,
    input wire sel,
    output wire out
);
    assign out = (sel == 0) ? in0 : in1;
endmodule
```

6) tb_mux2to1_1bit.v:

```
module tb_mux2to1;
    reg [3:0] in0, in1;
    reg sel;
    wire [3:0] out;

    mux2to1 uut (
        .in0(in0), .in1(in1), .sel(sel), .out(out)
    );

    initial begin
        $monitor("Time=%0t | sel = %b | in0 = %b | in1 = %b => out = %b", $time, sel, in0, in1, out);

        in0 = 4'b1100; in1 = 4'b0011;
        sel = 0; #10;
        sel = 1; #10;

        $finish;
    end
endmodule
```

---

7) dff4.v:

```
module dff4 (
    input wire clk,
    input wire reset, // Active-low
    input wire [3:0] data_in,
    output reg [3:0] data_out
);
    always @(posedge clk or negedge reset) begin
        if (!reset)
            data_out <= 4'b0000;
        else
            data_out <= data_in;
    end
endmodule
```

---

8) tb_dff4.v:

```
module tb_dff4;
    reg clk, reset;
    reg [3:0] data_in;
    wire [3:0] data_out;

    dff4 uut (
        .clk(clk), .reset(reset),
        .data_in(data_in),
        .data_out(data_out)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk; // Clock toggles every 5ns
    end

    initial begin
        reset = 0; data_in = 4'b0000; #10;
        reset = 1; data_in = 4'b1010; #10;
        data_in = 4'b0101; #10;
        reset = 0; #10;
        reset = 1; data_in = 4'b1111; #10;

        $finish;
    end
endmodule
```

9) full_adder_1bit.v:

```
module full_adder_1bit (
    input wire A, B, Cin,
    output wire Sum, Cout
);
    assign Sum = A ^ B ^ Cin;
    assign Cout = (A & B) | (B & Cin) | (A & Cin);
endmodule
```

---

10) tb_full_adder_1bit.v:

```
module tb_full_adder_1bit;
    reg A, B, Cin;
    wire Sum, Cout;

    full_adder_1bit uut (
        .A(A), .B(B), .Cin(Cin),
        .Sum(Sum), .Cout(Cout)
    );

    initial begin
        $display("A B Cin | Sum Cout");
        $monitor("%b %b %b | %b   %b", A, B, Cin, Sum, Cout);

        A=0; B=0; Cin=0; #10;
        A=0; B=1; Cin=0; #10;
        A=1; B=0; Cin=0; #10;
        A=1; B=1; Cin=0; #10;
        A=1; B=1; Cin=1; #10;

        $finish;
    end
endmodule
```

---

11) ripple_carry_adder_4bit.v:

```
module ripple_carry_adder_4bit (
    input wire [3:0] A, B,
    input wire Cin,
    output wire [3:0] Sum,
    output wire Cout
);
    wire [2:0] carry;

    full_adder_1bit FA0(A[0], B[0], Cin, Sum[0], carry[0]);
    full_adder_1bit FA1(A[1], B[1], carry[0], Sum[1], carry[1]);
    full_adder_1bit FA2(A[2], B[2], carry[1], Sum[2], carry[2]);
    full_adder_1bit FA3(A[3], B[3], carry[2], Sum[3], Cout);
endmodule
```

12) tb_ripple_carry_adder_4bit.v:

```
module tb_ripple_carry_adder_4bit;
    reg [3:0] A, B;
    reg Cin;
    wire [3:0] Sum;
    wire Cout;

    ripple_carry_adder_4bit uut (
        .A(A), .B(B), .Cin(Cin),
        .Sum(Sum), .Cout(Cout)
    );

    initial begin
        $monitor("Time=%0t | A=%b | B=%b | Cin=%b => Sum=%b | Cout=%b", $time, A, B, Cin, Sum, Cout);

        A = 4'd3; B = 4'd5; Cin = 0; #10;
        A = 4'd7; B = 4'd9; Cin = 0; #10;
        A = 4'd15; B = 4'd1; Cin = 0; #10;

        $finish;
    end
endmodule
```

---

13) adder_subtractort.v:

```
module adder_subtractor (
    input [3:0] A, B,
    input OP,
    output [7:0] S
);

    wire [3:0] B_muxed;
    wire [3:0] Sum;
    wire Cout;


    mux2to1 mux_B (
        .in0(B),
        .in1(~B),
        .sel(OP),
        .out(B_muxed)
    );


    ripple_carry_adder_4bit RCA (
        .A(A),
        .B(B_muxed),
        .Cin(OP),
        .Sum(Sum),
        .Cout(Cout)
    );


    assign S = {3'b000, Cout, Sum};

endmodule
```

14) tb_adder_subtractort.v:

```
module tb_adder_subtractor;
    reg [3:0] A, B;
    reg OP;
    wire [7:0] S;

    // Instantiate the Unit Under Test (UUT)
    adder_subtractor uut (
        .A(A),
        .B(B),
        .OP(OP),
        .S(S)
    );

    initial begin
        $monitor("Time=%0t | A=%d | B=%d | OP=%b | Result={%b} (Decimal: %d)",
                 $time, A, B, OP, S, S);

        // Test Addition
        OP = 0; A = 4'd7; B = 4'd5; #10;  // 7 + 5 = 12
        OP = 0; A = 4'd15; B = 4'd1; #10; // 15 + 1 = 16
        OP = 0; A = 4'd8; B = 4'd8; #10;  // 8 + 8 = 16

        // Test Subtraction (Positive Result)
        OP = 1; A = 4'd9; B = 4'd3; #10;  // 9 - 3 = 6

        // Test Subtraction (Negative Result)
        OP = 1; A = 4'd2; B = 4'd5; #10;  // 2 - 5 = -3 (Magnitude 3)

        // Test Subtraction (Equal Values)
        OP = 1; A = 4'd7; B = 4'd7; #10;  // 7 - 7 = 0

        $finish;
    end
endmodule
```

15) ALU.v:

```
module ALU (
    input wire clk,
    input wire reset,
    input wire [3:0] A, B,
    input wire [2:0] op_code,
    input wire [1:0] compute_op,
    output reg [7:0] result,
    output reg done, div_by_zero, negative
);
    always @(posedge clk or negedge reset) begin
        if (!reset) begin
            result <= 8'b0;
            done <= 0;
            div_by_zero <= 0;
            negative <= 0;
        end else begin
            done <= 0;
            div_by_zero <= 0;
            negative <= 0;
            case (op_code)
                3'b001: result <= {4'b0000, A}; // Load A
                3'b011: result <= {4'b0000, B}; // Load B
                3'b101: begin // Compute
                    case (compute_op)
                        2'b00: begin // ADD
                            result <= A + B;
                        end
                        2'b01: begin // SUB
                            if (A < B) begin
                                result <= B - A;
                                negative <= 1;
                            end else begin
                                result <= A - B;
                            end
                        end
                        2'b10: result <= A * B; // MUL
                        2'b11: begin // DIV
                            if (B == 0) begin
                                div_by_zero <= 1;
                                result <= 8'b0;
                            end else begin
                                result <= {A % B, A / B}; // {remainder, quotient}
                            end
                        end
                    endcase
                end
                3'b110: begin
                    done <= 1;
                end
                default: result <= result;
            endcase
        end
    end
endmodule
```

16) tb_ALU.v:

```
module tb_ALU;
    reg clk, reset;
    reg [3:0] A, B;
    reg [2:0] op_code;
    reg [1:0] compute_op;
    wire [7:0] result;
    wire done, div_by_zero, negative;

    ALU uut (
        .clk(clk), .reset(reset),
        .A(A), .B(B),
        .op_code(op_code), .compute_op(compute_op),
        .result(result), .done(done),
        .div_by_zero(div_by_zero), .negative(negative)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        reset = 0; #10;
        reset = 1;

        A = 4'd6; B = 4'd2; op_code = 3'b101; compute_op = 2'b00; #20; // Add
        A = 4'd6; B = 4'd9; op_code = 3'b101; compute_op = 2'b01; #20; // Sub
        A = 4'd4; B = 4'd3; op_code = 3'b101; compute_op = 2'b10; #20; // Mul
        A = 4'd8; B = 4'd2; op_code = 3'b101; compute_op = 2'b11; #20; // Div

        $finish;
    end
endmodule
```

17) Datapath.v:

```
module Datapath (
    input wire clk, reset,
    input wire [3:0] data_in,
    input wire wrt_addr,
    input wire [2:0] op_code,
    input wire [1:0] compute_op,
    output wire [7:0] result,
    output wire done, div_by_zero, negative
);
    wire [1:0] sel;
    wire [3:0] reg_A_out, reg_B_out;
    wire [3:0] reg_A_in, reg_B_in;

    // Decoder
    decoder_1to2 decoder_inst (
        .write_addr(wrt_addr),
        .sel(sel)
    );

    // Registers A and B
    dff4 reg_A (
        .clk(clk),
        .reset(reset),
        .data_in(data_in),
        .data_out(reg_A_out)
    );

    dff4 reg_B (
        .clk(clk),
        .reset(reset),
        .data_in(data_in),
        .data_out(reg_B_out)
    );

    // ALU
    ALU alu_inst (
        .clk(clk),
        .reset(reset),
        .A(reg_A_out),
        .B(reg_B_out),
        .op_code(op_code),
        .compute_op(compute_op),
        .result(result),
        .done(done),
        .div_by_zero(div_by_zero),
        .negative(negative)
    );
endmodule
```

18) tb_datapath.v:

```
module tb_datapath;
    reg clk, reset;
    reg [3:0] data_in;
    reg wrt_addr;
    reg [2:0] op_code;
    reg [1:0] compute_op;
    wire [7:0] result;
    wire done, div_by_zero, negative;

    Datapath uut (
        .clk(clk), .reset(reset), .data_in(data_in),
        .wrt_addr(wrt_addr), .op_code(op_code), .compute_op(compute_op),
        .result(result), .done(done), .div_by_zero(div_by_zero), .negative(negative)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        reset = 0; #10;
        reset = 1;

        // Load A = 5
        data_in = 4'd5; wrt_addr = 0; op_code = 3'b001; #10;

        // Load B = 3
        data_in = 4'd3; wrt_addr = 1; op_code = 3'b011; #10;

        // Compute Add
        op_code = 3'b101; compute_op = 2'b00; #20;

        // Compute Sub
        op_code = 3'b101; compute_op = 2'b01; #20;

        // Compute Mul
        op_code = 3'b101; compute_op = 2'b10; #20;

        // Compute Div
        op_code = 3'b101; compute_op = 2'b11; #20;

        $finish;
    end
endmodule
```

19) calculator_datapath.v:

```
module calculator_datapath(
    input wire clk,
    input wire reset,
    input wire [3:0] data_in,
    input wire wrt_addr,
    input wire [2:0] op_code,
    input wire [1:0] compute_op,
    output wire [7:0] result,
    output wire done,
    output wire div_by_zero,
    output wire negative
);

    wire [1:0] sel;
    wire [3:0] regA_in, regB_in;
    wire [3:0] regA_out, regB_out;

    decoder_1to2 my_decoder(
        .write_addr(wrt_addr),
        .sel(sel)
    );

    mux2to1 muxA_input (
        .in0(4'b0000),
        .in1(data_in),
        .sel(sel[0]),
        .out(regA_in)
    );

    mux2to1 muxB_input (
        .in0(4'b0000),
        .in1(data_in),
        .sel(sel[1]),
        .out(regB_in)
    );

    dff4 regA (
        .clk(clk),
        .reset(reset),
        .data_in(regA_in),
        .data_out(regA_out)
    );

    dff4 regB (
        .clk(clk),
        .reset(reset),
        .data_in(regB_in),
        .data_out(regB_out)
    );

    ALU alu_inst (
        .clk(clk),
        .reset(reset),
        .A(regA_out),
        .B(regB_out),
        .op_code(op_code),
        .compute_op(compute_op),
        .result(result),
        .done(done),
        .div_by_zero(div_by_zero),
        .negative(negative)
    );

endmodule
```

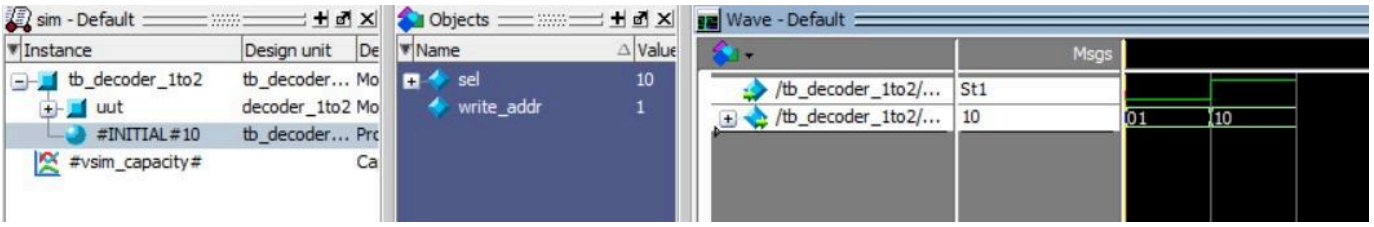
20) tb_calculator_datapath.v:

```
module tb_calculator_datapath;

    reg clk, reset;
    reg [3:0] data_in;
    reg wrt_addr;
    reg [2:0] op_code;
    reg [1:0] compute_op;
    wire [7:0] result;
    wire done;
    wire div_by_zero;
    wire negative;

    calculator_datapath uut (
        .clk(clk),
        .reset(reset),
        .data_in(data_in),
        .wrt_addr(wrt_addr),
        .op_code(op_code),
        .compute_op(compute_op),
        .result(result),
        .done(done),
        .div_by_zero(div_by_zero),
        .negative(negative)
    );


    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end


    initial begin

        reset = 0; #10;
        reset = 1; #10;


        // load A = 7
        wrt_addr = 0;  // select A
        data_in = 4'd7;
        op_code = 3'b001; // load A
        #10;
```

```
        // load B = 3
        wrt_addr = 1;  // select B
        data_in = 4'd3;
        op_code = 3'b011; // load B
        #10;


        // test addition
        op_code = 3'b101; // computing
        compute_op = 2'b00; // addition
        #20;


        // test subtraction
        op_code = 3'b101; // computing
        compute_op = 2'b01; // subtraction
        #20;


        // test multiplication
        op_code = 3'b101; // computing
        compute_op = 2'b10; // multiplication
        #20;


        // test division
        op_code = 3'b101; // computing
        compute_op = 2'b11; // division
        #20;


        // test division by 0
        // load A = 8
        wrt_addr = 0;
        data_in = 4'd8;
        op_code = 3'b001; // load a
        #10;

        // load B = 0
        wrt_addr = 1;
        data_in = 4'd0;
        op_code = 3'b011; // load b
        #10;

        // division part
        op_code = 3'b101; // computing
        compute_op = 2'b11; // division
        #20;

        $finish;
    end

endmodule
```

Our Simulation Results

1) decoder_1to2.v :




```
add wave -position insertpoint sim:/tb_decoder_1to2/uut/*
VSIM 3> run
# Time=0 | write_addr = 0 => sel = 01
VSIM 4> run
# Time=10 | write_addr = 1 => sel = 10
```

Explanation:

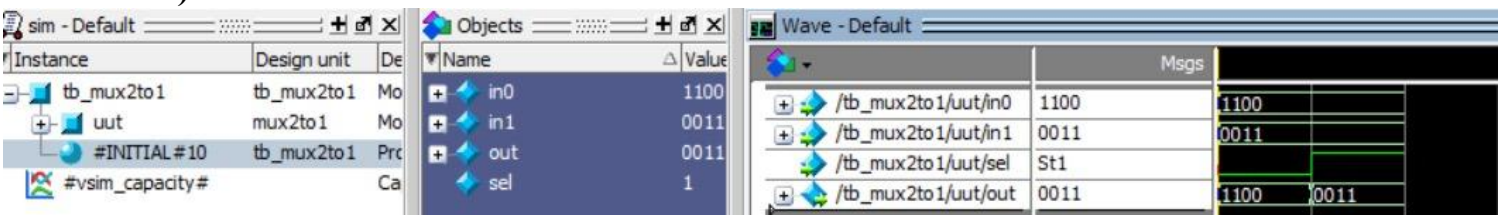
-when we write_addr is 0 the output sel becomes 01 meaning Register A is selected

-when we write_addr is 1 the output sel becomes 10 meaning Register B is selected

-so the waveform shows the decoder correctly translates the 1 bit input into a 2 bit output and behaves as expected for both possible input values

---

2) mux2to1.v :




```
add wave -position insertpoint sim:/tb_mux2to1/uut/*
VSIM 12> run
# Time=0 | sel = 0 | in0 = 1100 | in1 = 0011 => out = 1100
VSIM 13> run
# Time=10 | sel = 1 | in0 = 1100 | in1 = 0011 => out = 0011
```

Explanation:

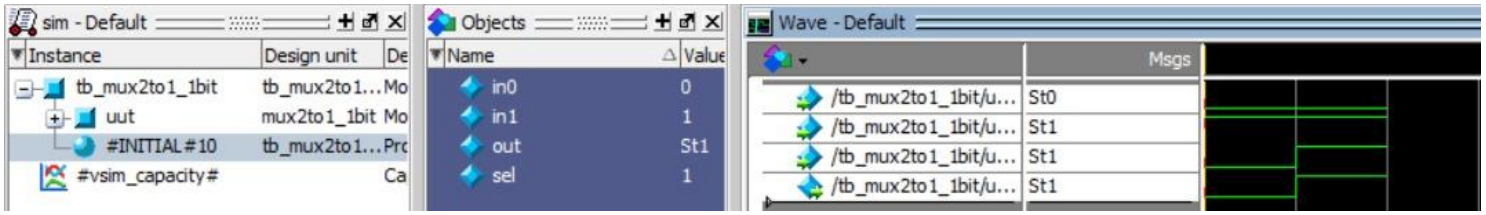
-at time 0 sel is 0 so the output follows in0 which is 1100

-at time 10 sel changes to 1 so the output switches to in1 which is 0011

-the waveform show that the multiplexer chooses between the two inputs according to the select line and the output changes immediately when the select input change

---

3) mux2to1_1bit.v :




```
add wave -position insertpoint sim:/tb_mux2to1_1bit/uut/*
VSIM 15> run
# Time=0 | sel = 0 | in0 = 0 | in1 = 1 => out = 0
VSIM 16> run
# Time=10 | sel = 1 | in0 = 0 | in1 = 1 => out = 1
```

Explanation:

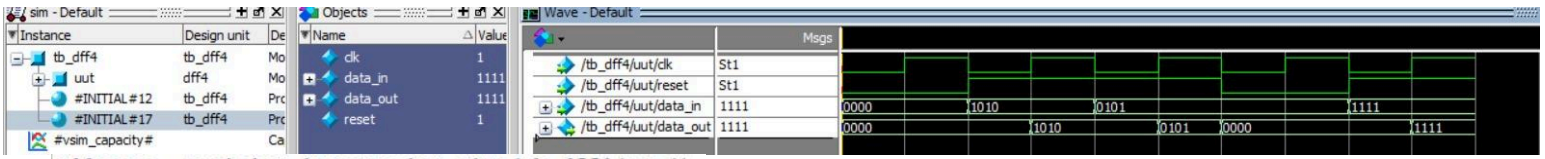
-at time 0 the select signal sel is 0 so the output follows in0 which is 0

-at time 10 sel changes to 1 so the output switches to in1 which is 1

-the waveform proves 1 bit multiplexer correctly changes its output based on the select input and that the output immediately follows the selected input

---

4) dff4.v :




```
add wave -position insertpoint sim:/tb_dff4/uut/*
VSIM 18> run
run
run
run
VSIM 19> run
```

Explanation:

-when reset is active low the output data_out becomes 0000 no matter what the input is

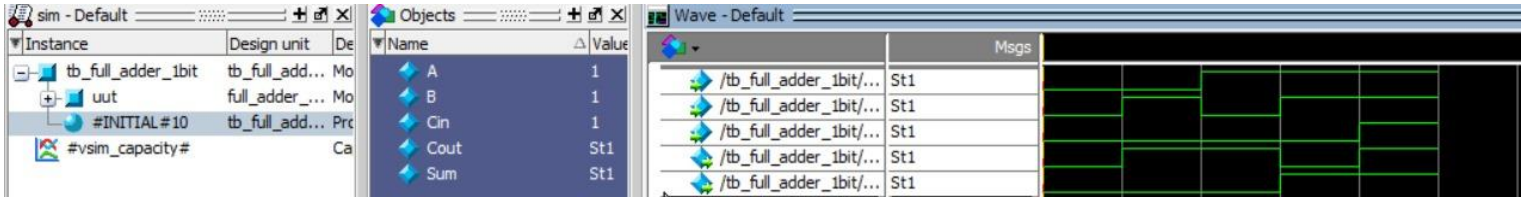
-when reset is active high the data_out follows data_in and updates its value at the rising edge of the clock

-we can see in the waveform that every time the clock rises data_out changes to match data_in if reset is high and if reset goes low data_out is cleared to 0000

-this proves that the register works correctly with synchronous data writing and asynchronous reset behavior

---

5) full_adder_1bit.v :




```
add wave -position insertpoint sim:/tb_full_adder_1bit/uut/*
VSIM 21> run
# A B Cin | Sum Cout
# 0 0 0 | 0   0
run
# 0 1 0 | 1   0
run
# 1 0 0 | 1   0
run
# 1 1 0 | 0   1
VSIM 22> run
# 1 1 1 | 1   1
```

Explanation:

-when A B and Cin are all zero both Sum and Cout are zero too

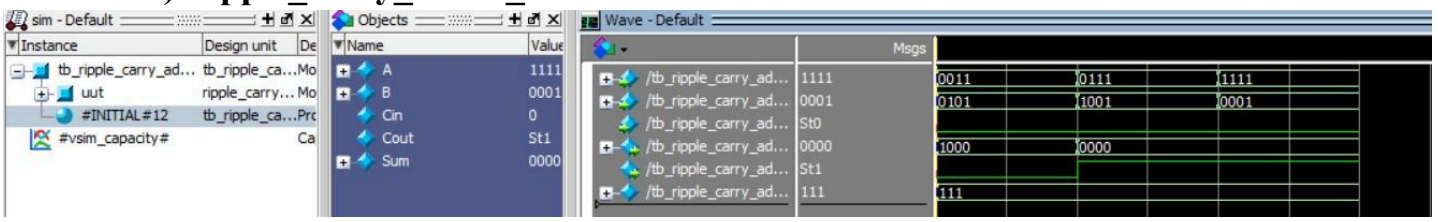
-when two of them are one and one is zero the Sum becomes zero and the Cout becomes one

-when all three inputs are one the Sum is one and the Cout is also one

-this shows that the full adder is adding the inputs correctly just like we learned

---

6) ripple_carry_adder_4bit.v :




```
add wave -position insertpoint sim:/tb_ripple_carry_adder_4bit/uut/*
VSIM 4> run
# Time=0 | A=0011 | B=0101 | Cin=0 => Sum=1000 | Cout=0
run
# Time=10 | A=0111 | B=1001 | Cin=0 => Sum=0000 | Cout=1
VSIM 5> run
# Time=20 | A=1111 | B=0001 | Cin=0 => Sum=0000 | Cout=1
```

Explanation:

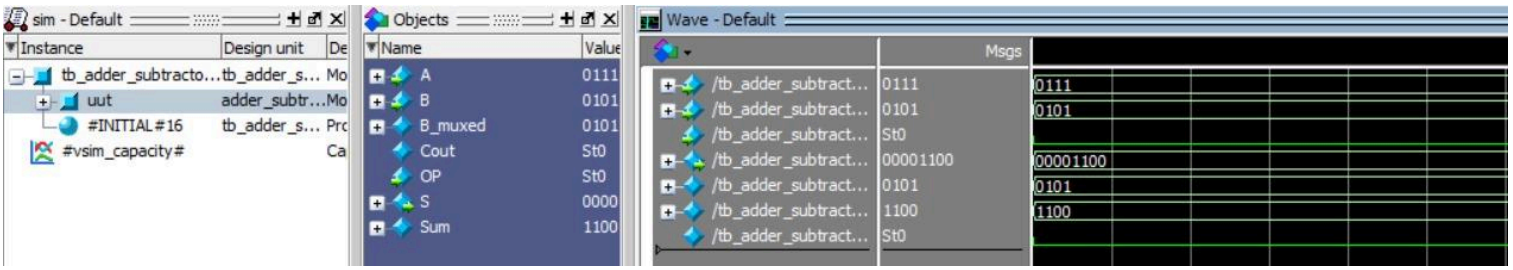
-first we add 0011 and 0101 and we get 1000 which is correct and there is no carry out

-then we add 0111 and 1001 and the sum becomes 0000 and we get a carry out which means it overflowed

-and we add 1111 and 0001 and again the sum is 0000 with a carry out

---

7) adder_subtractor.v :




```
add wave -position insertpoint sim:/tb_adder_subtractor/uut/*
VSIM 21> run
# Time=0 | A= 7 | B= 5 | OP=0 | Result={00001100} (Decimal:  12)
```

Explanation:

-at the start A is 7 and B is 5 and OP is 0 which means addition

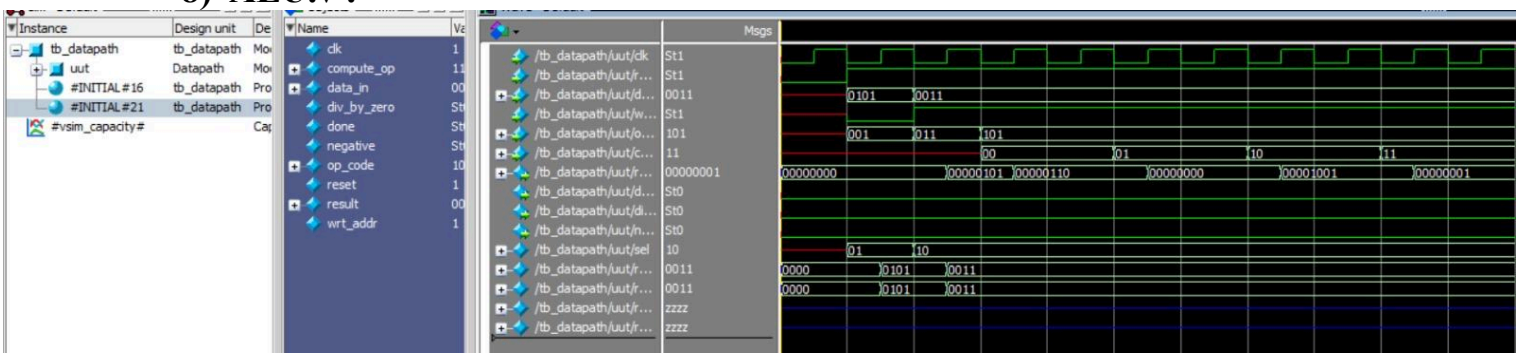
-the result is 12 which is correct for 7 plus 5

-we can see that the output S shows 00001100 which is 12 in decimal

-this shows that the adder_subtractor works properly for addition when OP is 0

---

8) ALU.v :



Explanation:

-first we made the ALU add two numbers and it gave the right answer

-then we made it subtract and it showed the correct result

-after that it multiplied and the number got bigger

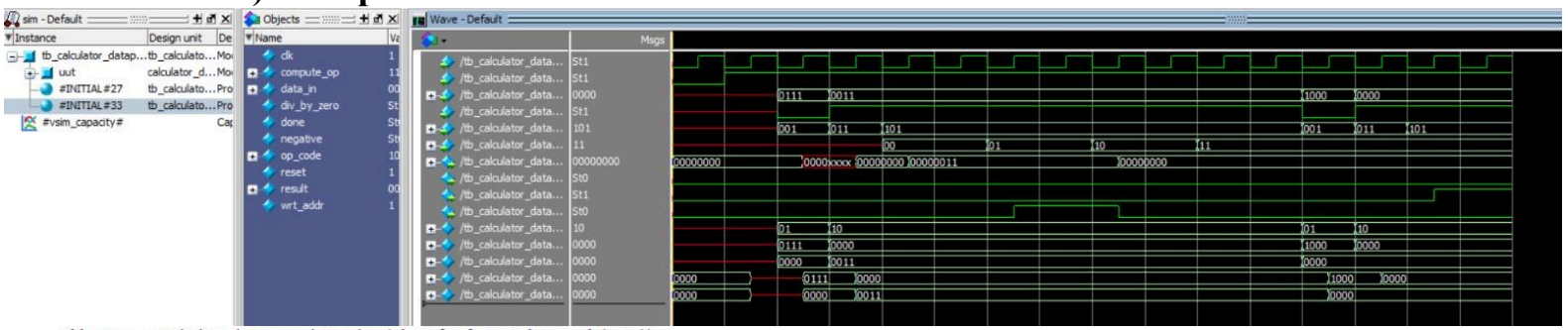
-at the end it divided and gave the right value

-the done signal also turned on when the ALU finished each job

-this shows that the ALU can add subtract multiply and divide like it is supposed to

```
add wave -position insertpoint sim:/tb_datapath/uut/*
VSIM 34> run
run
run
run
run
run
run
run
run
run
VSIM 35> run
```

9) Datapath.v :



```
add wave -position insertpoint sim:/tb_calculator_datapath/uut/*
VSIM 37> run
run
run
run
run
run
run
run
run
run
run
run
run
run
run
run
run
run
VSIM 38> run
```

Explanation:

-first we loaded numbers into the registers

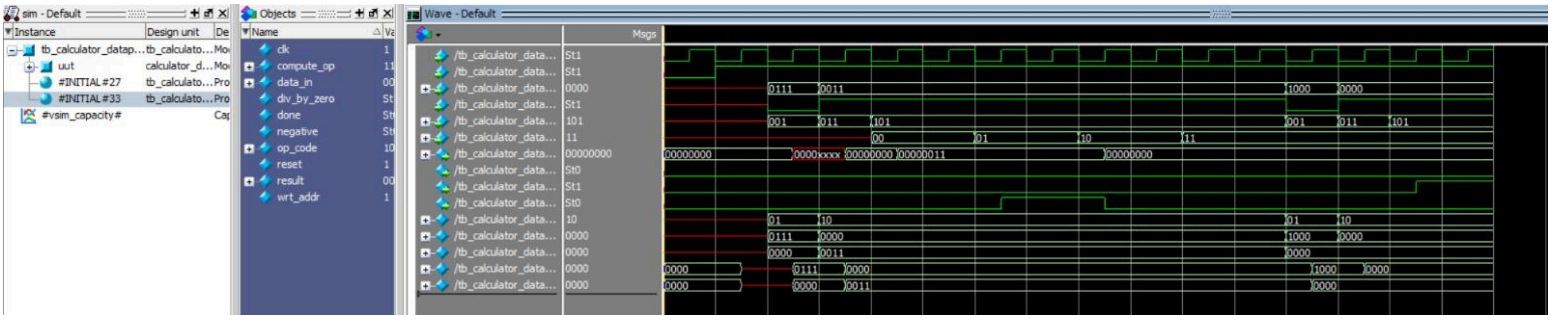
-then we chose which operation to do like add subtract multiply or divide

-the result changed every time based on what operation we picked

-we also saw the done signal turning on when the operation finished

-the waveform shows that the datapath is working right and all parts like decoder registers ALU are connected correctly and doing their jobs

10) calculator_datapath.v :



```
add wave -position insertpoint sim:/tb_calculator_datapath/uut/*
VSIM 4> run
run
run
run
run
run
run
run
run
run
run
run
run
run
run
run
run
run
VSIM 5> run
```

Explanation:

-first we put numbers into the calculator by loading them into the registers

-then we chose what operation we wanted like add subtract multiply or divide

-the result changed correctly each time when we gave a new command

-we also saw the done signal go high when the operation finish

Summary

In this project we learned how to design and simulate a 4 bit calculator using Verilog and ModelSim and we realized that even small mistakes in connections between modules can cause big problems in the final datapath

How can we improve ourselves

For future projects we can test every small module more carefully before trying to connect everything together in this project we learned that by working on smaller modules.

References

- Digital Design with An Introduction to the Verilog HDL, 5th Edition, M. Morris Mano & Michael D. Ciletti, Pearson